

# PROCEDIMENTO DE INSTALACAO E CONFIGURACAO DO AMBIENTE DE DESENVOLVIMENTO

## PIM - Projeto Integrado Multidisciplinar

Analise e Desenvolvimento de Sistemas - 2º Semestre

Disciplina: Programacao Estruturada em C

Windows 10/11 - 64 bits

| Componente                  | Padrao adotado                              |
|-----------------------------|---------------------------------------------|
| Editor                      | Visual Studio Code                          |
| Terminal de desenvolvimento | Prompt de Comando do Windows (cmd.exe)      |
| Gerenciador de pacotes      | MSYS2 / pacman                              |
| Toolchain                   | GCC / MinGW-w64 UCRT64                      |
| Ferramentas                 | GDB, Make e pkgconf                         |
| Banco relacional            | PostgreSQL Server + libpq                   |
| Banco NoSQL                 | MongoDB Community Server + MongoDB C Driver |
| Compilacao                  | Parametros explicitos do GCC                |

## 1. Objetivo

Este procedimento estabelece um ambiente unico para o desenvolvimento do PIM. O codigo sera escrito no Visual Studio Code e compilado no proprio Prompt de Comando do Windows, utilizando o GCC/MinGW-w64 UCRT64 instalado pelo MSYS2.

**Importante:** o MSYS2 UCRT64 sera utilizado principalmente para instalar e atualizar os pacotes por meio do pacman. O terminal cotidiano de desenvolvimento no VS Code sera o **cmd.exe**.

## 2. Arquitetura do ambiente

```
Windows 10/11
|
+-- Visual Studio Code
|   |
|   +-- Terminal integrado: cmd.exe
|       |
|       +-- gcc.exe
|       +-- gdb.exe
|       +-- mingw32-make.exe / make
|
+-- MSYS2
|   |
|   +-- UCRT64
|       +-- GCC / MinGW-w64
|       +-- GDB
|       +-- Make
|       +-- pkgconf
|       +-- libpq
|       +-- MongoDB C Driver
|
+-- PostgreSQL Server :5432
|
+-- MongoDB Community Server :27017
```

## 3. Ordem recomendada de instalacao

1. Instalar o Visual Studio Code e a extensao C/C++.
2. Instalar o MSYS2.
3. Atualizar o MSYS2.
4. Instalar GCC, GDB, Make e pkgconf no ambiente UCRT64.
5. Adicionar C:\msys64\ucrt64\bin ao PATH do Windows.
6. Configurar o Prompt de Comando como terminal padrao do VS Code.
7. Instalar PostgreSQL Server e pgAdmin 4.
8. Instalar MongoDB Community Server e MongoDB Compass.
9. Instalar libpq e MongoDB C Driver pelo MSYS2.
10. Executar testes de compilacao e conexao.

## 4. Instalacao do Visual Studio Code

Instale o Visual Studio Code para Windows 64 bits. Em seguida, abra Extensions e instale:

```
C/C++ - Microsoft
```

O VS Code sera utilizado para editar o codigo, organizar os arquivos do projeto e executar os comandos pelo terminal integrado.

## 5. Instalacao e atualizacao do MSYS2

Instale o MSYS2 preferencialmente no diretorio padrao:

```
C:\msys64
```

Para instalar e atualizar os pacotes, abra **MSYS2 UCRT64**.

## 5.1 Atualizacao

```
pacman -Syu
```

Caso seja solicitado o encerramento do terminal, feche-o, abra novamente o MSYS2 UCRT64 e repita:

```
pacman -Syu
```

## 6. Instalacao da toolchain UCRT64

No MSYS2 UCRT64, execute:

```
pacman -S --needed \  
mingw-w64-ucrt-x86_64-gcc \  
mingw-w64-ucrt-x86_64-gdb \  
mingw-w64-ucrt-x86_64-make \  
mingw-w64-ucrt-x86_64-pkgconf
```

Esses pacotes instalam o compilador GCC, o depurador GDB, o Make e o pkgconf dentro de:

```
C:\msys64\ucrt64
```

## 7. Configuracao do PATH do Windows

Para que o Prompt de Comando e o VS Code encontrem diretamente gcc.exe e as DLLs do ambiente UCRT64, adicione ao PATH do Windows:

```
C:\msys64\ucrt64\bin
```

### 7.1 Procedimento

- Abra o menu Iniciar e pesquise por Variaveis de Ambiente.
- Abra Editar as variaveis de ambiente do sistema.
- Clique em Variaveis de Ambiente.
- Em Path, escolha Editar.
- Adicione C:\msys64\ucrt64\bin.
- Confirme todas as janelas.
- Feche e abra novamente o VS Code para que o novo PATH seja carregado.

### 7.2 Verificacao no Prompt de Comando

```
where gcc  
gcc --version  
  
where gdb  
gdb --version
```

O comando **where gcc** deve localizar, preferencialmente:

```
C:\msys64\ucrt64\bin\gcc.exe
```

Se outro GCC aparecer antes desse caminho, revise o PATH para evitar mistura de toolchains.

## 8. Terminal do VS Code

O terminal padrao para desenvolvimento sera o proprio **Command Prompt / Prompt de Comando**.

No VS Code, utilize:

```
Terminal  
> New Terminal  
> Command Prompt
```

Se desejar defini-lo como padrao, abra a paleta de comandos com Ctrl+Shift+P, escolha **Terminal: Select Default Profile** e selecione **Command Prompt**.

### 8.1 Teste

```
where gcc
gcc --version
```

Não é necessário abrir o terminal MSYS2 UCRT64 para compilar os programas do PIM.

## 9. Primeiro programa em C

```
#include <stdio.h>

int main(void)
{
    printf("Ambiente PIM configurado com sucesso!\n");
    return 0;
}
```

No Prompt de Comando integrado do VS Code:

```
gcc main.c -o main.exe
main.exe
```

A partir deste ponto, os exemplos utilizarão a sintaxe de caminhos do Windows.

## 10. Instalação do PostgreSQL Server

Instale o PostgreSQL Server para Windows. Mantenha PostgreSQL Server, pgAdmin 4 e Command Line Tools.

| Parametro              | Padrao sugerido |
|------------------------|-----------------|
| Host                   | localhost       |
| Porta                  | 5432            |
| Usuario administrativo | postgres        |
| Banco do PIM           | pim             |

A senha do usuário postgres será definida durante a instalação. Não publique credenciais reais no repositório ou no código entregue.

### 10.1 Verificação

Abra **services.msc** e confirme que o serviço PostgreSQL está em execução.

### 10.2 Criação do banco

No pgAdmin, conecte-se ao servidor e execute:

```
CREATE DATABASE pim;
```

## 11. Instalação do MongoDB Community Server

Instale o MongoDB Community Server para Windows como serviço. Recomenda-se instalar também o MongoDB Compass.

| Parametro      | Padrao sugerido           |
|----------------|---------------------------|
| Host           | localhost                 |
| Porta          | 27017                     |
| URI local      | mongodb://localhost:27017 |
| Banco sugerido | pim                       |

### 11.1 Verificação

Abra **services.msc** e confirme que o serviço MongoDB está em execução. Depois, no Compass, teste:

```
mongodb://localhost:27017
```

## 12. Instalação da libpq para C

Volte ao terminal MSYS2 UCRT64 somente para instalar o pacote:

```
pacman -S --needed mingw-w64-ucrt-x86_64-postgresql
```

A biblioteca libpq permite que o programa em C se comunique com o PostgreSQL.

## 13. Compilacao C + PostgreSQL por parametros

```
#include <stdio.h>
#include <libpq-fe.h>

int main(void)
{
    printf("libpq carregada corretamente.\n");
    printf("Versao: %d\n", PQlibVersion());
    return 0;
}
```

No Prompt de Comando do VS Code, compile utilizando parametros explicitos:

```
gcc teste_postgresql.c ^
-IC:\msys64\ucrt64\include ^
-LC:\msys64\ucrt64\lib ^
-lpq ^
-o teste_postgresql.exe
```

No cmd.exe, o caractere ^ permite continuar o mesmo comando na linha seguinte. O comando tambem pode ser escrito em uma unica linha.

### 13.1 Significado dos parametros

- **-I:** informa o diretorio dos arquivos de cabecalho.
- **-L:** informa o diretorio das bibliotecas.
- **-lpq:** vincula o programa com a biblioteca PostgreSQL libpq.
- **-o:** define o nome do executavel gerado.

Execucao:

```
teste_postgresql.exe
```

## 14. Teste C -> PostgreSQL

```
#include <stdio.h>
#include <libpq-fe.h>

int main(void)
{
    PGconn *conn = PQconnectdb(
        "host=localhost port=5432 dbname=pim user=postgres password=SUA_SENHA"
    );

    if (PQstatus(conn) != CONNECTION_OK)
    {
        fprintf(stderr, "Erro: %s\n", PQerrorMessage(conn));
        PQfinish(conn);
        return 1;
    }

    printf("Conexao com PostgreSQL realizada com sucesso!\n");
    PQfinish(conn);
    return 0;
}
```

```
gcc teste_postgresql.c ^
-IC:\msys64\ucrt64\include ^
-LC:\msys64\ucrt64\lib ^
-lpq ^
-o teste_postgresql.exe
```

```
teste_postgresql.exe
```

## 15. Instalacao do MongoDB C Driver

Abra o MSYS2 UCRT64 apenas para instalar:

```
pacman -S --needed mingw-w64-ucrt-x86_64-mongo-c-driver
```

O pacote instala o driver C do MongoDB e a biblioteca BSON dentro da árvore UCRT64.

## 16. Identificacao dos diretorios do MongoDB C Driver

Como os nomes dos diretorios de include podem conter o numero da versao instalada, verifique-os no Windows.

No Prompt de Comando:

```
dir /b C:\msys64\ucrt64\include\mongoc*
dir /b C:\msys64\ucrt64\include\bson*
```

Exemplo de resultado:

```
mongoc-2.2.1
bson-2.2.1
```

Use nos comandos de compilacao o nome efetivamente existente no computador.

## 17. Compilacao C + MongoDB por parametros

```
#include <stdio.h>
#include <mongoc/mongoc.h>

int main(void)
{
    mongoc_init();
    printf("MongoDB C Driver carregado com sucesso!\n");
    mongoc_cleanup();
    return 0;
}
```

Exemplo, considerando a versao 2.2.1:

```
gcc teste_mongodb.c ^
-IC:\msys64\ucrt64\include\mongoc-2.2.1 ^
-IC:\msys64\ucrt64\include\bson-2.2.1 ^
-LC:\msys64\ucrt64\lib ^
-lmongoc2 ^
-lbson2 ^
-o teste_mongodb.exe

teste_mongodb.exe
```

### 17.1 Significado dos parametros

- **-I...mongoc...**: diretorio dos cabecalhos do MongoDB C Driver.
- **-I...bson...**: diretorio dos cabecalhos BSON.
- **-L...**: diretorio das bibliotecas UCRT64.
- **-lmongoc2**: vincula o driver MongoDB.
- **-lbson2**: vincula a biblioteca BSON.
- **-o**: define o executavel.

## 18. Teste C -> MongoDB

```
#include <stdio.h>
#include <mongoc/mongoc.h>

int main(void)
{
    mongoc_client_t *client;
    bson_t command = BSON_INITIALIZER;
    bson_t reply;
    bson_error_t error;

    mongoc_init();
```

```

client = mongoc_client_new("mongodb://localhost:27017");
BSON_APPEND_INT32(&command, "ping", 1);

if (!mongoc_client_command_simple(
    client, "admin", &command, NULL, &reply, &error))
{
    fprintf(stderr, "Erro: %s\n", error.message);
    bson_destroy(&command);
    mongoc_client_destroy(client);
    mongoc_cleanup();
    return 1;
}

printf("Conexao com MongoDB realizada com sucesso!\n");

bson_destroy(&reply);
bson_destroy(&command);
mongoc_client_destroy(client);
mongoc_cleanup();

return 0;
}

```

```

gcc teste_mongodb.c ^
-IC:\msys64\ucrt64\include\mongoc-2.2.1 ^
-IC:\msys64\ucrt64\include\bson-2.2.1 ^
-LC:\msys64\ucrt64\lib ^
-lmongoc2 ^
-lbson2 ^
-o teste_mongodb.exe

teste_mongodb.exe

```

## 19. Papel do pkgconf

O pkgconf permanece instalado porque pode ser utilizado para consultar a instalacao das bibliotecas. Entretanto, a abordagem didatica adotada no PIM e a **compilacao por parametros explicitos**.

Assim, os alunos visualizam diretamente os parametros **-I**, **-L**, **-l** e **-o**, em vez de receber automaticamente os parametros de compilacao por substituicao de comandos.

## 20. Instalacao completa dos pacotes MSYS2

Depois que o MSYS2 estiver atualizado, os pacotes podem ser instalados de uma vez no terminal UCRT64:

```

pacman -S --needed \
mingw-w64-ucrt-x86_64-gcc \
mingw-w64-ucrt-x86_64-gdb \
mingw-w64-ucrt-x86_64-make \
mingw-w64-ucrt-x86_64-pkgconf \
mingw-w64-ucrt-x86_64-postgresql \
mingw-w64-ucrt-x86_64-mongo-c-driver

```

## 21. IntelliSense no VS Code

Na pasta do projeto, crie **.vscode/c\_cpp\_properties.json**:

```

{
  "configurations": [
    {
      "name": "GCC UCRT64",
      "compilerPath": "C:/msys64/ucrt64/bin/gcc.exe",
      "includePath": [
        "${workspaceFolder}/**",
        "C:/msys64/ucrt64/include/**"
      ],
      "cStandard": "c17",
      "intelliSenseMode": "windows-gcc-x64"
    }
  ],
}

```

```
"version": 4
}
```

## 22. Estrutura sugerida para o PIM

```
pim/
|
+-- src/
|   +-- main.c
|   +-- usuario.c
|   +-- cliente.c
|   +-- produto.c
|   +-- relatorio.c
|
+-- include/
|   +-- usuario.h
|   +-- cliente.h
|   +-- produto.h
|   +-- relatorio.h
|
+-- database/
|   +-- postgres.c
|   +-- postgres.h
|   +-- mongodb.c
|   +-- mongodb.h
|
+-- .vscode/
|
+-- Makefile
```

## 23. Makefile e o Prompt de Comando

O Makefile pode ser introduzido posteriormente para automatizar a compilacao. Como o pacote GNU Make instalado pertence ao ambiente UCRT64, confirme no Prompt de Comando qual executavel foi disponibilizado:

```
where make
where mingw32-make
```

Para alunos do segundo semestre, a compilacao manual por parametros pode ser utilizada inicialmente, permitindo compreender claramente cada etapa antes da automatizacao.

## 24. Ambiente local e arquitetura do PIM

DESENVOLVIMENTO LOCAL

VS Code + cmd.exe

|  
v

Programa C

|  
+--> localhost:5432 --> PostgreSQL  
|  
+--> localhost:27017 --> MongoDB

ARQUITETURA DO PIM

Estacao Cliente Windows

Programa C

|  
+----- LAN -----> Servidor PostgreSQL :5432  
|  
+----- LAN -----> Servidor MongoDB :27017

Quando os bancos forem executados em servidores da rede, o principio permanece o mesmo. O endereco **localhost** sera substituido pelo hostname ou endereco IP correspondente ao servidor.

## 25. Checklist final

■ Windows 10/11 64 bits.

- Visual Studio Code instalado.
- Extensao C/C++ instalada.
- MSYS2 instalado e atualizado.
- Pacotes UCRT64 instalados.
- C:\msys64\ucrt64\bin adicionado ao PATH.
- Prompt de Comando selecionado como terminal do VS Code.
- where gcc localiza o GCC UCRT64.
- gcc --version funciona no cmd.exe.
- PostgreSQL Server esta em execucao.
- Banco pim foi criado.
- MongoDB Server esta em execucao.
- Compass conecta em localhost:27017.
- libpq esta instalada.
- MongoDB C Driver esta instalado.
- Programa C simples compila no cmd.exe.
- Programa PostgreSQL compila com parametros explicitos.
- Programa C conecta ao PostgreSQL.
- Programa MongoDB compila com parametros explicitos.
- Programa C executa ping no MongoDB.

## 26. Resumo do fluxo de trabalho

```

INSTALACAO / ATUALIZACAO
MSYS2 UCRT64
|
+--> pacman

DESENVOLVIMENTO
Visual Studio Code
|
v
Prompt de Comando (cmd.exe)
|
v
gcc arquivo.c [parametros] -o programa.exe
|
v
programa.exe

```

**Padrao oficial do laboratorio:** MSYS2 UCRT64 para gerenciamento dos pacotes e **Prompt de Comando do Windows para compilacao e execucao no VS Code.**